Document Retrieval Assignment Report

COM3110: Text Processing

Yejin Kang

The aim of this project is to implement a document retrieval system in a Vector Space Model (VCM) using different term-weighting schemes and preprocessing techniques and to evaluate the performance affected by the use of different weighting schemes and preprocessing techniques using standard evaluation metrics. The implemented system retrieves top 10 ranked documents that are most relevant to each user query.

The system supports two optional preprocessing techniques: stoplisting and stemming. Stoplisting removes common non-informative words, while stemming reduces the words to their root form to increase the likelihood of matching between queries and documents. Four preprocessing configurations were evaluated: no preprocessing, stoplisting only, stemming-only and both stoplisting and stemming combined. The system also implements three term weighting schemes, binary, term frequency (TF), and term frequency inverse document frequency (TF-IDF).

The code implementation begins by computing core statistical components during system initialisation: the number of documents in the collection (N), document frequency (DF), inverse document frequency (IDF) and document vector magnitudes ($|\vec{d}|$). These are computed using the functions get_df, get_idf and get_doc_vec_length, respectively. The function get_df determines the number of documents in which each term appears. The IDF values are then computed using $IDF = log(\frac{N}{DF})$. Finally, document vector magnitude is computed as $|\vec{d}| = \sqrt{d_1^2 + d_2^2 + d_3^2 ... d_N^2}$. Document term weights are calculated according to the selected term weighting option (-w). In binary weighting, the weight is set to 1, indicating the presence of the term in a document. In TF weighting, the weight is given by the raw TF value, and in TF-IDF weighting, the weight is computed as $TF \times IDF$.

Query processing, converting user queries into query vectors, is performed by the get_query_vector function by assigning a weight to each query term according to the selected term weighting scheme. The function also handles the case where a query term does not exist in the document collection by assigning it a zero IDF weight, ensuring that only shared terms contribute to the similarity score.

The similarity between a query and the document collection is computed using cosine similarity between the query and each document vector within the for_query function. A set of candidate documents is first constructed, containing only documents that include at least one query term. The document term weight is then computed according to the selected term weighting scheme. Then, the dot product between the query vector and the document vector is calculated by multiplying them and accumulated to a variable. Only positive dot products, the similarity scores, are retained and these scores are divided by the corresponding document vector magnitudes to obtain the final cosine similarity values. The query vector magnitude is omitted from the calculation as it is constant for a given query. The resulting scores are sorted in descending order, and the top 10 most relevant document IDs are returned for each query.

To improve efficiency, DF, IDF, and document vector magnitudes are precomputed once during system initialisation, rather than being recalculated for each query. During retrieval, only candidate documents were scored, which significantly reduces unnecessary computing and improves scalability for larger document collections compared to scoring all documents. The use of IDF further improves performance by reducing the influence of highly frequent terms and emphasising rare, more discriminative terms within the documents collection.

The system was evaluated using a gold standard relevance file as ground truth. Figure 1 and figure 2 illustrate the resulting precision, recall and F-measure values. The best performing term weighting scheme is TF-IDF, as it consistently outperformed the other two schemes across all evaluation metrics and preprocessing configurations. The superior performance of TF-IDF proves its

ability to downweight highly frequent terms and emphasise rare, more informative terms. Binary performs the worst as it ignores term frequency completely.

Applying both stemming and stoplisting yields the highest performance for all term weighting schemes, indicating that removing non-informative stopwords and reducing words to their root forms significantly improves retrieval effectiveness. In contrast, configuration without preprocessing consistently produced the lowest scores as irrelevant stopwords introduce noise and morphological variations reduce matching effectiveness.

When only one preprocessing technique is applied, binary and TF performed better under stoplisting than under stemming, while TF-IDF showed stronger performance with stemming than stoplisting, suggesting its greater benefit from root-based term matching.

Term weighting	Stemming(-p)	Stoplisting(-s)	Precision	Recall	F-measure
Binary	Off	Off	0.07	0.06	0.06
	On	Off	0.09	0.08	0.08
	Off	On	0.13	0.10	0.11
	On	On	0.16	0.13	0.15
TF	Off	Off	0.08	0.06	0.07
	On	Off	0.11	0.09	0.10
	Off	On	0.17	0.13	0.15
	On	On	0.19	0.15	0.17
TF-IDF	Off	Off	0.21	0.17	0.18
	On	Off	0.26	0.21	0.23
	Off	On	0.22	0.18	0.19
	On	On	0.27	0.22	0.24

Figure 1. Result table

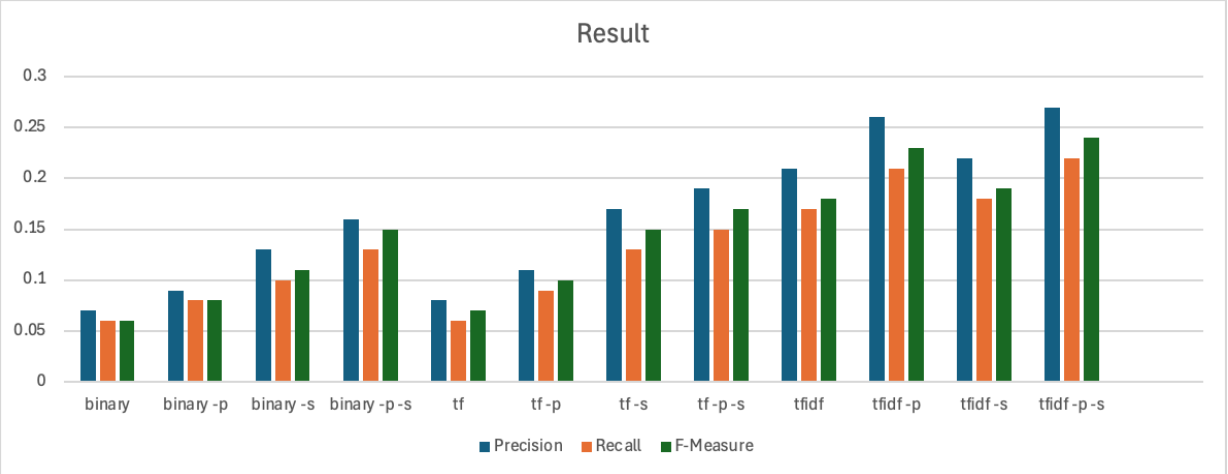

Figure 2. Result Chart